tmux documentation

Native dual-OS Plan — `tmx` works flawlessly on Linux AND macOS with host access + isolation

> Engineering plan for: making `tmx` deliver three things simultaneously on both Linux and macOS — (1) plain-vanilla-tmux UX, (2) full host command access, (3) per-session resource isolation. Captures the architectural decisions made 2026-05-13 by the operator after observing that the VM-based design failed requirement (2).
>
> **No implementation lands without this plan reviewed and the four-gate regression suite GREEN at the end.**

§0 — Why this plan supersedes the VM design

Requirement (operator-mandated)	VM design (current)	Native dual-OS (this plan)
Plain tmux UX	✓ (after a fix cycle)	✓
Per-session resource cap	✓ Linux cgroup, in VM	✓ Linux cgroup native, ✓ macOS rlimit native
Full host command access (Homebrew, system tools, `scutil`, etc.)	✗ session is `core@localhost` in VM	✓ session is the operator's user on the host
OOM in session A doesn't kill B	✓ on VM	✓ Linux cgroup scope, ✓ macOS rlimit per-process
Built reproducibly from pinned source	✓	✓

The VM design satisfied (1)+(2)+(4) but failed (3) — operator could not run `brew`, `scutil`, `lima`, Mach-O binaries from inside a tmx session. The native dual-OS design satisfies all five by delivering the strongest containment each OS supports natively.

§1 — Architecture (one paragraph)

`tmx new -s NAME` on **Linux** spawns a fresh tmux server on socket `tmx-NAME` inside a fresh cgroup-v2 transient scope `tmx-NAME.scope` via `systemd-run --user --scope`. Identical to current default-arch but the binary is the Linux ELF run **on the operator's Linux host**, not in any VM. The shell inside the session is the operator's `$SHELL`, with full PATH, full FS, full host commands.

`tmx new -s NAME` on **macOS** spawns a fresh tmux server on socket `tmx-NAME` with the operator's `$SHELL` wrapped in a `ulimit`-set launcher: `ulimit -v <bytes> -t <cpu-seconds> -u <max-procs>; exec $SHELL -l`. RLIMIT_AS, RLIMIT_CPU, RLIMIT_NPROC are enforced by the Darwin kernel per-process; children inherit. Shell PATH includes `/opt/homebrew/bin`, `/usr/local/bin`, all of the operator's environment. `brew`, `scutil`, `osascript`, Homebrew Mach-O binaries — all reachable.

The binary is **built natively for each OS** from the same `tmux/` submodule (pinned to upstream tag `3.6a`):

- Linux: `scripts/build_containerized.sh` → `tmux/build/bin/tmux` (Linux ELF, existing pipeline)
- macOS: `scripts/build_native.sh` → `tmux/build-darwin/bin/tmux` (Mach-O, NEW pipeline, native compile against Homebrew libevent + jemalloc)

The wrapper picks the right binary based on `uname -s` and the right isolation mechanism based on the same.

§2 — Per-OS isolation primitives

§2.1 Linux — cgroup-v2 transient scopes (unchanged)

```
systemd-run --user --scope \
    --unit=tmx-NAME.scope \
    -p MemoryMax=<host-adaptive>  \
    -p CPUQuota=200%              \
    -p TasksMax=4096              \
    -p Delegate=yes               \
    tmux -L tmx-NAME new-session -d -s NAME
```

Tests 09, 14, 15 already prove this works (PASS=15/0/0 destructive suite GREEN). Constitution §11.4.7 operator-path coverage already binds. No change to Linux-side test gates.

§2.2 macOS — POSIX rlimit wrapper per session shell

```
tmux -L tmx-NAME new-session -d -s NAME \
    "/path/to/scripts/tmx-rlimit-wrapper $TMX_MEM_BYTES $TMX_CPU_SEC $TMX_
```

`scripts/tmx-rlimit-wrapper` (NEW, ~10 lines):

```
#!/usr/bin/env bash
# Wraps the operator's shell with kernel-enforced resource
        limits.
# Inherited by all children. Per-session enforcement is per-
        process
# in the limit hierarchy — see docs/guide/README.md §5.6 for the
        strength
# comparison with Linux cgroup.
mem_bytes="$1"; shift
cpu_sec="$1"; shift
proc_max="$1"; shift
ulimit -v "$mem_bytes" 2>/dev/null   # RLIMIT_AS (virtual memory)
ulimit -t "$cpu_sec"   2>/dev/null   # RLIMIT_CPU
ulimit -u "$proc_max"  2>/dev/null   # RLIMIT_NPROC
exec "$@"
```

Verification per §11.4.2 (captured runtime evidence): `tmux send-keys -t NAME 'ulimit -v -t -u' Enter` then `tmux capture-pane -p` returns the actual rlimit values from the kernel. Numeric readback matches configured cap.

§2.3 The strength gap (honest documentation per §1)

Linux cgroup is **per-group** and **kernel-tracked**: every process in the scope counts toward `memory.current`, kernel OOM-kills the scope when total exceeds `memory.max`, no process escapes.

macOS rlimit is **per-process** and **inherited**: each forked process has its own RLIMIT_AS, but if all 100 children individually respect their per-process cap and TOTAL across them exceeds the practical host memory, the host swaps / pressures. This is weaker than cgroup.

The plan documents this in `README.md` Architecture section and `docs/guide/README.md` §5.6 explicitly. No bluff. On Linux you get per-group containment; on macOS you get per-process containment. Both are the strongest the OS supports natively.

§3 — Build pipelines

§3.1 Linux (unchanged)

`scripts/build_containerized.sh` — already works. Produces `tmux/build/bin/tmux` (Linux ELF, jemalloc-linked, `-fstack-protector-strong`, etc.).

§3.2 macOS (NEW)

`scripts/build_native.sh`:

```
1. Verify Homebrew + required brews present:
   libevent, jemalloc, automake, autoconf, pkg-config
   (Auto-install if missing AND user is on macOS — `brew install` is safe.

2. cd tmux/ (the submodule)
3. autoreconf -fi  (if configure absent)
4. ./configure \
     --prefix="$PWD/../tmux/build-darwin" \
     CFLAGS="-O2 -DNDEBUG -fstack-protector-strong -D_FORTIFY_SOURCE=2 \
             -I$(brew --prefix)/include" \
     LDFLAGS="-Wl,-bind_at_load \
              -Wl,-search_paths_first \
              -L$(brew --prefix)/lib \
              -ljemalloc"
5. make -j$(sysctl -n hw.ncpu)
6. make install
7. Verify binary: tmux/build-darwin/bin/tmux -V → "tmux 3.6a"
   Verify Mach-O: file tmux/build-darwin/bin/tmux | grep "Mach-O"
   Verify jemalloc linked: otool -L tmux/build-darwin/bin/tmux | grep -q j
```

Output: `tmux/build-darwin/bin/tmux` Mach-O binary, arm64 or x86_64 per `arch -arm64` / `arch -x86_64` invocation. Universal binary is future work; ship native-arch first.

§3.3 The verification gate per OS

`scripts/verify.sh` already respects `$TMUX_BIN` env var. Setup.sh picks the right binary path based on `uname -s` before invoking verify.

§4 — Wrapper rewrite (OS-aware)

`scripts/tmx.template` — single template, OS-dispatched at runtime.

```
case "$(uname -s)" in
    Linux)
        # systemd-run --user --scope ... (existing
        implementation)
        ;;
    Darwin)
        # rlimit wrapper (new implementation)
        ;;
    *)
        # SKIP — unsupported OS, fall back to plain tmux
        invocation
        ;;
esac
```

The host-adaptive memory calculation, session-name sanitisation, collision detection, and hostname-colour application are OS-agnostic (work on both). Only the isolation-mechanism dispatch is per-OS.

§5 — Tests per OS

Test	Purpose	Linux behaviour	macOS behaviour
01 smoke	binary version	runs natively	runs natively
02 session	tmux lifecycle	native	native
03 jemalloc	LD_PRELOAD / DYLD_INSERT_LIBRARIES	LD_PRELOAD	DYLD_INSERT_LIBRARIES + `DYLD_FORCE_FLAT_NAMESPACE`
04 history-limit	config respected	native	native
05 clear-history	apparent leak fix	native	native
06 concurrent panes	RSS bounded	native	native
07 long session	leak check	native	native
08 oom_score_adj	-500 set	tmx-oom-set via setcap	SKIP (no oom_score_adj on Darwin)
09 crash isolation	cgroup scope ops	systemd-run scope	SKIP (no systemd on Darwin)
10 hostname colour	algorithm	works on both	works on both
11 hostname colour integration	wrapper applies bg	works on both	works on both

Test	Purpose	Linux behaviour	macOS behaviour
12 memory pressure	OOM in scope	systemd-run + stress-ng	NEW: ulimit -v + memory allocation; verify malloc fails
13 TasksMax	fork-bomb	cgroup pids.max	NEW: ulimit -u; verify fork() returns EAGAIN
14 concurrent OOM independence	sessions B+C survive A's OOM	cgroup independence	NEW: rlimit independence (different sessions, different ulimits)
15 per-session cgroup	distinct units	cgroup readback	NEW: per-session ulimit readback via send-keys + capture-pane

The Linux destructive tests (12/13/14) and Test 15 stay as-is on Linux. macOS equivalents are NEW tests using ulimit-based verification. Test 09 (cgroup-scope-specific) SKIPs on Darwin with explicit reason per §11.4.3 topology dispatch.

Meta-test mutations M1–M10 are mostly OS-agnostic (algorithm-level). M4, M5, M9, M10 (wrapper-targeting) need per-OS sed patterns; the target file is now `scripts/tmx` on the host (no more `scripts/tmx-vm` since there's no VM in daily use).

§6 — Files changed

File	Change
`scripts/build_native.sh`	NEW — macOS native build
`scripts/build_containerized.sh`	UNCHANGED (still used for Linux artifact)
`scripts/tmx.template`	OS-aware dispatch in scope-creation block
`scripts/tmx-rlimit-wrapper.sh`	NEW — wraps shell with ulimit on macOS
`scripts/setup.sh`	OS-aware: native build on Darwin, containerized on Linux; no VM bridge
`scripts/tmx-mac.template`	DELETE (no more bridge — tmx runs natively on macOS)
`scripts/tmx-vm`	DELETE (no more VM-side wrapper for daily use)
`scripts/test_vm.sh`	KEEP for CI cross-verification only; remove from default operator flow
`scripts/test_e2e.sh`	rewritten to test the NATIVE flow (no bridge)
`scripts/tests/12_*.sh`	OS-aware: rlimit on macOS, cgroup on Linux
`scripts/tests/13_*.sh`	OS-aware
`scripts/tests/14_*.sh`	OS-aware
`scripts/tests/15_*.sh`	OS-aware
`scripts/tests/16_macos_rlimit_distinct.sh`	NEW — macOS-specific per-session rlimit verification
`README.md`	architecture diagram redrawn for native dual-OS
`docs/guide/README.md`	§5.6 updated to describe both isolation primitives

File	Change
`Constitution.md`	§11.4.3 topology dispatch now covers Darwin natively
`CLAUDE.md` / `AGENTS.md`	updated to reference native dual-OS

§7 — Acceptance criteria

The plan is "done" when ALL of these are true:

1. `bash scripts/setup.sh` on Darwin completes GREEN, produces `tmux/build-darwin/bin/tmux` (Mach-O), installs the shell snippet pointing at the host-local wrapper (no VM bridge).
2. `tmx new -s X` on Darwin opens a session whose prompt is `$USER@<mac-hostname>` (NOT `core@localhost`), with full PATH including Homebrew, with `which brew` returning a path.
3. `tmx new -s heavy` on Darwin with `TMX_MEM=2G` enforces a 2 GiB virtual-memory cap on the session shell; `ulimit -v` inside returns `2097152`.
4. `bash scripts/setup.sh` on a Linux host (existing path) still GREEN — `TMX_TEST_DESTRUCTIVE=1 bash scripts/verify.sh` PASS=15 FAIL=0 SKIP=0.
5. Meta-test 10/10 mutations caught on Linux; macOS-equivalent mutations caught on macOS.
6. `Constitution.md` §11.4.7 still bind; no test bypasses the operator path.

§8 — Migration / regression risk

- Operators who already ran the VM-based setup will have a stale bridge in `scripts/tmx`. Re-running setup.sh overwrites it with the native wrapper. The VM remains startable for CI verification only.
- The `scripts/tmx-vm` file is removed from default flow but kept as `scripts/test_vm.sh` for CI cross-verification of the Linux artifact on a real Linux VM.
- Existing sessions running in the podman machine VM will need to be killed (`podman machine ssh "<vm-path>/scripts/tmx-vm kill-server"`). After that the VM can be stopped and forgotten by daily operators.